REPASO FINAL · LUNES ANTES DEL EXAMEN

# Lo que te queda, bien explicado

Cada paso con su comando y lo que te tiene que salir.
Sin saltos, sin suponer nada.

**Bloques A–H**
Workflows, filtros, euro.py, Gitflow, Pages, needs/env/if, matrix, secrets, custom actions

**Examen final simulacro**
Todo junto, como el de verdad, con solución

Regla de oro: **un paso → mira que sale lo del cuadro verde → siguiente.**
Si no sale lo verde, para y pásame el error.

PLAN DE HOY

# Orden y tiempos

| # | Bloque | ⏱ |
|---|---|---|
| **0** | Antes de empezar: limpiar y reglas anti-lío | 5' |
| **A** | Workflow desde cero (repaso de la 16) | 15' |
| **B** | Filtros: cuándo se ejecuta un workflow | 20' |
| **C** | Práctica euro.py (la del profe) | 40' |
| 🍿 | **1 episodio** | |
| **D** | Gitflow: feature, release y hotfix | 25' |
| **E** | GitHub Pages en tu repo web | 15' |
| **F** | Varios jobs: needs, env, if | 20' |
| **G** | matrix y secrets | 20' |
| **H** | Custom Actions (Docker) | 30' |
| 🍿 | **1 episodio** | |
| 👹 | Examen final simulacro (con la chuleta, como el real) | 60' |

**🎯 Si vas justo de tiempo**

Haz en este orden: **0 → C → D → E → Examen final**. Lo demás es extra.

**💡 Cómo es el examen (lo que sabemos)**

Práctico, en la máquina virtual. Corrige mirando el `git log`. GitHub solo al final para **entregar** (push + tags). Con apuntes: lleva la chuleta.

BLOQUE 0

# Antes de empezar

⏱ 5 min | 🎯 Dejar ~/git limpio y no volver a liarla

## 1 ¿Quedó un repo suelto en ~/git?

```
ls -a ~/git
```

✅ Salen tus carpetas ( `web` , `flow` , `pruebas-actions` ...) y **NO** sale `.git` . → Pasa al bloque A.

❌ Si sale `.git` o `.github` sueltos en `~/git` , bórralos (es el repo que creaste sin querer):

```
rm -rf ~/git/.git ~/git/.github ~/git/README.md
```

## Las 4 reglas anti-lío

**1 · Primero la carpeta, luego el init**

`mkdir nombre` → `cd nombre` → `git init` . Nunca `git init` en `~/git` ni en `~` .

**2 · Mira el prompt antes de cada comando**

Te dice **carpeta** y **rama**: `~/git/web [master` . Si ves `…36` o `…38` estás en el repo de la home de la máquina: no estás donde crees.

### 3 · Nunca crees un fichero importante con touch

`touch` crea ficheros **vacíos**. Para escribir contenido: `cat > fichero << 'EOF'` … `EOF`, o `nano fichero`. Comprueba siempre con `cat fichero`.

### 4 · Commit antes de cambiar de rama

`git status` limpio → `git switch`. Lo que no tenga commit viaja contigo a la otra rama.

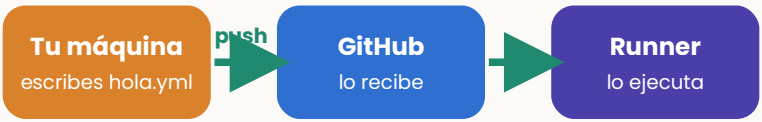
BLOQUE A

# Workflow desde cero

⏱ 15 min

🎯 Hacerlo sin mirar, de carpeta vacía a ✅

Un **workflow** es un fichero **.yml** dentro de la carpeta **.github/workflows/** de tu repo. Cuando haces **push**, GitHub lo lee y ejecuta sus pasos en una máquina suya. El resultado se ve en la pestaña **Actions**.




Resultado → pestaña Actions del repo (✅ verde / ❌ rojo)

## El esqueleto, línea a línea

```yaml
# nombre que verás en Actions
name: Primer workflow
# CUÁNDO: en cada push
on: [push]
# lista de trabajos
jobs:
  # nombre del trabajo (lo inventas)
  saludar:
    # en qué máquina
    runs-on: ubuntu-latest
    # los pasos, en orden
    steps:
      - name: Bajar mi código
        # uses = usar una action ya hecha
        uses: actions/checkout@v4
      - name: Decir hola
        # run = un comando de terminal
        run: echo "Hola"
```

⚠️ **Sangría**

2 espacios por nivel, **nunca Tab**. Cada step empieza con `-` (guion + espacio). `name`, `uses`, `run` del mismo step van alineados.

## Ejercicios

### 🟢 A1 · Repo nuevo con workflow (sin mirar, solo chuleta R9)

1. ☐ En GitHub crea el repo vacío `repaso-actions`.
2. ☐ En `~/git`: carpeta `repaso-actions`, entra, `git init`, rama `main`.
3. ☐ Crea `.github/workflows/hola.yml` con el esqueleto (con `cat > … << 'EOF'`). Compruébalo con `cat`.
4. ☐ Commit, `remote add`, `push -u`. Mira en Actions que sale "Hola".

### 🟡 A2 · Modificar y volver a subir

1. ☐ Con `nano`, añade un step con `run: |` que haga `date` y `ls -la`.
2. ☐ `git commit -am` + `git push`. En Actions sale una ejecución **nueva**.

Soluciones → SA

BLOQUE B

# Cuándo se ejecuta: filtros

⏱ 20 min

🎯 Entender por qué un workflow se ejecuta (o no) en una rama

La parte **on:** dice **cuándo** se lanza el workflow. Lo importante para el examen: **en qué ramas**. Si tu rama no está en la lista, al hacer push **no pasa nada** en Actions.

| Escribes | Se ejecuta… |
|---|---|
| `on: [push]` | en push a **cualquier** rama |
| `on:`<br>`  push:`<br>`  branches: [ main ]` | solo en push a **main** |
| `branches: [ main, test ]` | en push a main **o** test |
| añadir `workflow_dispatch:` | además, con un **botón** "Run workflow" en Actions |
| `pull_request:` | cuando se abre una PR |

💡 **LA CLAVE (entiende esto y tienes euro.py)**

Cada rama tiene **su propia copia** del fichero .yml. Cuando haces push a la rama `test`, GitHub lee el .yml **que hay en test**. Así que: si en `test` pones `branches: [ main, test ]`, en `test` se ejecuta. Y lo que cambies en el .yml de `test` no toca el de `main`.

## 1 Limitar el workflow a main

En `~/git/repaso-actions` (del bloque A):

```
cd ~/git/repaso-actions
nano .github/workflows/hola.yml
```

Cambia la línea `on: [push]` por estas 3:

```
on:
  push:
    branches: [ main ]
```

```
git commit -am "Solo en main" && git push
```

✅ En Actions aparece una ejecución nueva (porque has hecho push a main).

## 2 Probar en otra rama: NO se ejecuta

```
git switch -c prueba
echo "x" >> README.md && git add README.md
git commit -m "Cambio en prueba"
git push -u origin prueba
```

✅ En Actions **NO** aparece ejecución nueva. Correcto: el .yml de `prueba` dice "solo main".

## 3 Añadir la rama al filtro: ahora SÍ

```
# branches: [ main, prueba ]
nano .github/workflows/hola.yml
git commit -am "Añade prueba al filtro" && git
push
```

✅ Ahora sí sale una ejecución, y a la izquierda pone la rama **prueba**.

## 4 Botón manual

Añade `workflow_dispatch:` dentro de `on:`, a la misma altura que `push:`:

```
on:
  push:
    branches: [ main, prueba ]
  workflow_dispatch:
```

```
git commit -am "Añade botón manual" && git push
```

✅ Actions → clic en "Primer workflow" (izquierda) → botón **Run workflow** → elige rama → Run.

El botón solo aparece si el .yml con `workflow_dispatch` está en la rama principal (main). Si no lo ves, haz merge de prueba en main y push.

### 🟡 B1 · Ejercicio

1. ☐ Crea la rama `develop` desde main. Haz push. ¿Se ejecuta? ¿Por qué?
2. ☐ Haz que se ejecute en `develop` tocando el .yml **solo en develop**. Push. Compruébalo.
3. ☐ Vuelve a main y mira el .yml con `cat`: ¿tiene develop? ¿Por qué no?

Soluciones → SB

BLOQUE C

# Práctica euro.py

⏱ 40 min

🎯 La práctica del profe entera, entendiendo cada paso

El profe tiene un repo con un script de Python (**euro.py**) que muestra el cambio de monedas, y un workflow que lo ejecuta. Tú tienes que: **1)** clonarlo, **2)** subirlo a TU GitHub, **3)** instalar dependencias con requirements.txt, **4)** rama **test** con Python 3.12, **5)** rama **development** que ejecute solo **USD**.

## Qué hay en el repo

| Fichero | Qué es |
|---|---|
| `euro.py` | El script. `python euro.py` = todas las monedas. `python euro.py USD` = solo dólar. |
| `requirements.txt` | Lista de librerías que necesita, una por línea ( `requests` , `pandas` ). |
| `.github/workflows/actions.yml` | El workflow "Monedes": instala Python, las librerías y ejecuta el script. Solo en **main**. |

## 1 Clonar el repo del profe

```
cd ~/git
git clone https://github.com/titansmc/actions.git
cd actions
ls -a
```

✅ Ves `euro.py`, `requirements.txt`, `README.md`, `.github`. Prompt: `~/git/actions [main`.

❌ "destination path 'actions' already exists": ya lo clonaste antes. Usa `cd actions` y sigue.

## 2 Mira el workflow para entenderlo

```
cat .github/workflows/actions.yml
```

```
name: Monedes
on:
  push:
    # ↓ solo main
    branches: [ main ]
jobs:
  create-list:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout del repo
        uses: actions/checkout@v4
      - name: Configurem python
        # ↓ instala Python
        uses: actions/setup-python@v7
        with:
          # ↓ qué versión
          python-version: '3.12'
      - name: Preparem l'entorn amb els paquets
        # ↓ instala librerías
        run: pip install -r requirements.txt
      - name: Correm l'script
        # ↓ ejecuta el script
        run: python ./euro.py
```

### 3 Subirlo a TU GitHub

En GitHub crea el repo `actions` **vacío**. Luego:

```
# ahora apunta al del profe
git remote -v
git remote set-url origin https://github.com/
fabiangorbaranaquino/actions.git
# ahora al tuyo
git remote -v
git push -u origin main
```

¿Por qué **set-url** y no **add**? Porque al clonar, `origin` ya existe (apuntando al profe). Solo le cambias la dirección.

✅ Actions de tu repo → "Monedes" en ✅ verde. Abre el step **Correm l'script**: sale una tabla de monedas.

❌ "refusing to allow a Personal Access Token... workflow": token nuevo con **repo** + **workflow**.

## 4 Tarea 3: requirements.txt

```
cat requirements.txt
```

El profe ya lo dejó hecho: el workflow usa `pip install -r requirements.txt`, que instala **todo lo que haya en ese fichero**, línea a línea. Para demostrarlo, añade una librería:

```
echo "numpy" >> requirements.txt
git commit -am "Añade numpy a requirements"
git push
```

✅ En Actions, step **Preparem l'entorn**: aparece "Successfully installed ... numpy".

## 5 Tarea 4: rama test con Python 3.12

```
git switch -c test
nano .github/workflows/actions.yml
```

Haz **2 cambios** en el fichero:

```
    # ↓ 1. añade test
    branches: [ main, test ]
...
          # ↓ 2. 3.12 (con comillas)
          python-version: '3.12'
```

Y para poder comprobarlo, añade al final un step más (misma sangría que los otros):

```
      - name: Ver versión
        run: python --version
```

```
git commit -am "Rama test con Python 3.12"
git push -u origin test
```

✅ En Actions sale una ejecución en la rama **test**. El step "Ver versión" dice **Python 3.12.x**.

❌ Si no sale ninguna ejecución: te faltó añadir `test` a `branches`.

## 6 Tarea 5: rama development, solo USD

```
# development sale de main
git switch main
git switch -c development
nano .github/workflows/actions.yml
```

2 cambios:

```
    # ↓ 1. añade development
    branches: [ main, development ]
...
        # ↓ 2. el último run, con USD
        run: python ./euro.py USD
```

```
git commit -am "Development ejecuta solo USD"
git push -u origin development
```

✅ Ejecución en la rama **development**. El step "Correm l'script" muestra **solo USD**.

## 7 Comprobar todo

```
git log --oneline --graph --all
```

```
* c3 (HEAD -> development, origin/development)
Development ejecuta solo USD
| * c2 (origin/test, test) Rama test con Python
3.12
|/
* c1 (origin/main, main) Añade numpy a
requirements
* ...commits del profe...
```

✅ 3 ramas, cada una con su commit, y en GitHub → Actions: ejecuciones en main, test y development, todas ✅.

### 🟡 C1 · Ejercicio extra

1. ☐ Crea la rama `gbp` desde main que ejecute solo `GBP` con Python `'3.11'`. Push y comprueba.
2. ☐ En `development`, haz que muestre `USD` y `JPY` a la vez.

Soluciones → SC

## 🍿 RECOMPENSA

**1 EPISODIO**

☐ Bloques A, B y C hechos
☐ euro.py con 3 ramas en ✅

BLOQUE D

# Gitflow: feature, release, hotfix

⏱ 25 min

🎯 Hacerlo a mano y que el log quede perfecto

Gitflow son **normas** para usar ramas, no comandos nuevos. Todo se hace con **switch**, **merge --no-ff**, **tag** y **branch -d**, que ya sabes.

| Rama | Para qué | Sale de | Vuelve a |
|---|---|---|---|
| **master** | lo publicado | — | — |
| **develop** | donde se trabaja | master | — |
| **feature/x** | algo nuevo | develop | develop |
| **release/ 1.0.0** | publicar versión | develop | master **+ tag** y develop |
| **hotfix/1.0.1** | bug urgente | **master** | master **+ tag** y develop |

💡 **El truco para no liarte nunca**

Antes de cada `git merge`, mira el prompt y pregúntate: **"¿estoy en la rama que RECIBE?"**. El merge trae la otra rama **a donde estás**.

⚡ **Merge sin que se abra nano**

`git merge --no-ff rama -m "Mensaje"` → con `-m` no se abre el editor.

## 1 Repo nuevo con master

```
mkdir -p ~/git/flow2 && cd ~/git/flow2
git init && git branch -m master
echo "# Tienda" > Readme.md
git add Readme.md && git commit -m "Commit
inicial"
```

✅ Prompt: `~/git/flow2 [master`.

## 2 develop sale de master

```
git switch -c develop master
```

✅ Prompt: `[develop`.

### 3 Feature: empezar, trabajar, cerrar

```
# empezar (sale de develop)
git switch -c feature/login develop
# trabajar
echo "<h1>Login</h1>" > login.html
git add login.html && git commit -m
"Añade página de login"
# cerrar: vuelve a develop
git switch develop
git merge --no-ff feature/login -m "Merge
feature/login"
git branch -d feature/login
```

✅ `git log --oneline --graph --all` → una "burbuja" en develop. master sigue en "Commit inicial".

## 4 Release 1.0.0: publicar

```
# empezar (sale de develop)
git switch -c release/1.0.0 develop
echo "1.0.0" > VERSION
git add VERSION && git commit -m
"Prepara versión 1.0.0"
# cerrar: 1) a master + tag
git switch master
git merge --no-ff release/1.0.0 -m "Merge
release/1.0.0"
git tag -a 1.0.0 -m "Versión 1.0.0"
#         2) a develop
git switch develop
git merge --no-ff release/1.0.0 -m "Merge
release/1.0.0 en develop"
#         3) borrar
git branch -d release/1.0.0
```

✅ `ls` en master: `Readme.md login.html VERSION`.
`git tag` → `1.0.0`.

## 5 Hotfix 1.0.1: bug urgente

```
# empezar (sale de MASTER, ¡no de develop!)
git switch -c hotfix/1.0.1 master
echo "<p>Arreglado</p>" >> login.html
git commit -am "Arregla bug del login"
# cerrar: 1) a master + tag
git switch master
git merge --no-ff hotfix/1.0.1 -m "Merge hotfix/
1.0.1"
git tag -a 1.0.1 -m "Versión 1.0.1"
#         2) a develop
git switch develop
git merge --no-ff hotfix/1.0.1 -m "Merge hotfix/
1.0.1 en develop"
#         3) borrar
git branch -d hotfix/1.0.1
```

## 6 Comprobar

```
git log --oneline --graph --all --decorate
git branch          # solo develop y master
git tag             # 1.0.0 y 1.0.1
```

✅ Solo quedan **master** y **develop**. Los tags **tag: 1.0.0** y **tag: 1.0.1** están en commits de merge de master. Todo son "burbujas" por el `--no-ff`.

### 🟡 D1 · Ahora tú solo (con la chuleta, sección 8)

1. ☐ En `flow2`: feature `feature/carrito` con `carrito.html`, cerrada en develop.
2. ☐ Release `1.1.0` con su tag.
3. ☐ Hotfix `1.1.1` con su tag.

BLOQUE E

# GitHub Pages en tu repo web

⏱ 15 min

🎯 La práctica "Github Pages web" sin mirar

GitHub Pages publica una web **gratis** a partir de una rama de tu repo. Lo típico: una rama llamada **gh-pages** con **index.html** (la portada) y **style.css**. Cada **push** a esa rama actualiza la web sola.

## 1 Repo limpio

```
cd ~/git/web
git status
git remote -v
```

✅ "nothing to commit" (o solo untracked como `.env`) y `origin` apunta a tu repo `web` de GitHub.

❌ Si hay "Changes not staged/to be committed": `git add .` + `git commit -m "Guarda cambios"`. Si dice MERGING: `git merge --abort`.

### 2 Rama gh-pages

```
git switch -c gh-pages
```

❌ "already exists" → `git switch gh-pages`.

### 3 Los 2 ficheros (copia y pega cada bloque entero)

```
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Mi web</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Hola, soy Fabián</h1>
  <p>Mi web en GitHub Pages</p>
</body>
</html>
EOF
```

```
cat > style.css << 'EOF'
body { font-family: sans-serif; background:
#f4f4f4; }
h1 { color: purple; }
EOF
```

La línea `<link rel="stylesheet" href="style.css">` es la que **"enlaza"** el CSS. Sin ella, el CSS existe pero no se aplica.

```
# comprobar que no están vacíos
cat index.html style.css
```

### 4 Commit y push de la rama

```
git add index.html style.css
git commit -m "Publica web con estilos"
git push -u origin gh-pages
```

### 5 Activar Pages (en la web de GitHub)

Repo **web** → **Settings** → menú izquierdo **Pages** → Source: **Deploy from a branch** → Branch: **gh-pages** y **/ (root)** → **Save**.

✅ Pestaña **Actions**: "pages build and deployment" en ✅ (1-2 min). Abre `https://fabiangorbaranaquino.github.io/web/` con Ctrl+F5: título morado.

### 6 Actualizar la web

```
# cambia el texto del <h1>
nano index.html
git commit -am "Cambia el título"
git push
```

✅ Espera el ✅ en Actions, Ctrl+F5: título nuevo. Sin tocar nada en GitHub.

BLOQUE F

# Varios jobs: needs, env, if

⏱ 20 min

🎯 Entender orden de jobs, variables y condiciones

| Palabra | Qué hace | Ejemplo |
|---|---|---|
| `needs` | el job espera a otro | `needs: build` |
| `env` | variable para usar con `$NOMBRE` | `APP: tienda` |
| `if` | solo se ejecuta si se cumple | `if: github.ref_name == 'main'` |
| `${{ github.ref_name }}` | nombre de la rama | `echo "${{ github.ref_name }}"` |

💡 **Sin needs, los jobs van a la vez**

Con `needs` se ponen en fila. Si uno falla, los que lo necesitan **no se ejecutan**.

## 1 Nuevo workflow en repaso-actions

```
cd ~/git/repaso-actions && git switch main
cat > .github/workflows/jobs.yml << 'EOF'
name: Varios jobs
on: [push]
env:
  APP: tienda
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Compilando $APP"
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Probando $APP en la rama $
{{ github.ref_name }}"
  deploy:
    needs: test
    if: github.ref_name == 'main'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Desplegando $APP"
EOF
git add .github && git commit -m "Añade workflow
con varios jobs" && git push
```

✅ Actions → "Varios jobs": un dibujo con **build → test → deploy** en fila, los 3 en ✅.

### 2 El if en otra rama

```
git switch -c otra
echo "y" >> README.md && git commit -am "Cambio
en otra"
git push -u origin otra
```

✅ build y test ✅, **deploy aparece gris (skipped)**: no es main.

### 3 needs cuando algo falla

En la rama `otra`, cambia el step de build por `- run: exit 1` (con nano). Commit y push.

✅ build ❌ rojo, test y deploy **no se ejecutan**. Así funciona `needs`.

Después vuelve a main: `git switch main`.

BLOQUE G

# matrix y secrets

⏱ 20 min

🎯 Un job que se repite solo y una contraseña escondida

**matrix** = ejecutar el **mismo job varias veces** con valores distintos (varios sistemas, varias versiones de Python...) sin copiarlo. **secrets** = guardar una contraseña en GitHub para usarla en el workflow **sin que se vea** en el código ni en los logs.

## matrix: cómo se lee

```yaml
strategy:
  matrix:
    # 2 valores
    os: [ubuntu-latest, windows-latest]
    # 2 valores
    python: ['3.11', '3.12']
# usa el valor de esta vuelta
runs-on: ${{ matrix.os }}
```

🎯 **Cuántos jobs salen**

**Multiplica**: 2 sistemas × 2 versiones = **4 jobs**. GitHub los lanza todos a la vez.

💡 **fail-fast**

Por defecto (`true`): si uno falla, cancela los demás. Con `fail-fast: false` siguen todos.

### 1 Workflow con matrix

```
cd ~/git/repaso-actions && git switch main
cat > .github/workflows/matrix.yml << 'EOF'
name: Matrix
on: [push]
jobs:
  probar:
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
        python: ['3.11', '3.12']
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python }}
      - run: python --version
      - run: echo "Sistema ${{ runner.os }}"
EOF
git add .github && git commit -m "Añade matrix"
&& git push
```

✅ Actions → "Matrix" → a la izquierda **4 jobs**: probar (ubuntu, 3.11), (ubuntu, 3.12), (windows, 3.11), (windows, 3.12). Cada uno enseña su versión y sistema.

## secrets: paso a paso

### 2 Crear el secreto en GitHub (web)

Repo **repaso-actions** → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Name: `MI_CLAVE` · Secret: `1234` → **Add secret**.

### 3 Usarlo en un workflow

```
cat > .github/workflows/secreto.yml << 'EOF'
name: Secreto
on: [push]
jobs:
  usar:
    runs-on: ubuntu-latest
    steps:
      - name: Usar la clave
        env:
          CLAVE: ${{ secrets.MI_CLAVE }}
        run: echo "La clave es $CLAVE"
EOF
git add .github && git commit -m "Usa un
secreto" && git push
```

✅ En el log sale **La clave es ***. GitHub la tapa: así no se filtra nunca.

Se mete en una variable con `env:` (dentro del step) y se usa con `$CLAVE`.

**🟡 G1 · Ejercicio**

1. ☐ Cambia la matrix a 3 versiones de Python (`'3.11', '3.12', '3.13'`) y solo ubuntu. ¿Cuántos jobs salen?
2. ☐ Crea el secreto `USUARIO_BD` y muéstralo junto a `MI_CLAVE` en el mismo step.

Solución → SG

BLOQUE H

# Custom Actions (Docker)

⏱ 30 min

🎯 Crear tu propia action y usarla en un workflow

Hasta ahora usabas actions de otros (`actions/checkout`, `actions/setup-python`). Una **custom action** es una **tuya**. La forma más fácil es con **Docker**: son **3 ficheros** en la raíz de un repo.

| Fichero | Qué es |
|---|---|
| `action.yml` | La "ficha" de la action: nombre, descripción, qué recibe (**inputs**), qué devuelve (**outputs**) y cómo se ejecuta (**runs**). |
| `Dockerfile` | Qué máquina (contenedor) se monta y qué script ejecuta. |
| `entrypoint.sh` | El script que hace el trabajo. Lo que recibe va en `$1`, `$2`... |

**🎯 Teoría que cae**

· En `action.yml` son obligatorios: **name**, **description**, **runs**.

· Docker: solo Linux, más lenta. JavaScript: todos los sistemas, más rápida.

· Un input `who-to-greet` crea la variable `INPUT_WHO-TO-GREET`.

## 1 Repo nuevo

En GitHub crea `hola-action` vacío.

```
mkdir -p ~/git/hola-action && cd ~/git/hola-
action
git init && git branch -m main
```

## 2 action.yml

```
cat > action.yml << 'EOF'
name: 'Hola'
description: 'Saluda y devuelve la hora'
inputs:
  who-to-greet:
    description: 'A quién saludar'
    required: true
    default: 'World'
outputs:
  time:
    description: 'La hora del saludo'
runs:
  using: 'docker'
  image: 'Dockerfile'
  args:
    - ${{ inputs.who-to-greet }}
EOF
```

`args` pasa el input al script: le llegará como `$1`.

### 3 Dockerfile

```
cat > Dockerfile << 'EOF'
FROM alpine:3.10
COPY entrypoint.sh /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
EOF
```

Monta un Linux pequeño (alpine), copia tu script y lo ejecuta al arrancar.

### 4 entrypoint.sh (¡y hacerlo ejecutable!)

```
cat > entrypoint.sh << 'EOF'
#!/bin/sh -l
echo "Hola $1"
echo "time=$(date)" >> $GITHUB_OUTPUT
EOF
chmod +x entrypoint.sh
```

La última línea del script es cómo se **devuelve** un output (`time`). Sin `chmod +x` falla con "permission denied".

## 5 Workflow que usa tu action

```
mkdir -p .github/workflows
cat > .github/workflows/usar.yml << 'EOF'
name: Usar mi action
on: [push]
jobs:
  saludo:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - id: hola
        uses: ./
        with:
          who-to-greet: 'Fabián'
      - run: echo "Hora del saludo $
{{ steps.hola.outputs.time }}"
EOF
```

| Línea | Por qué |
|---|---|
| `actions/checkout` | sin él, el runner no tiene tus ficheros y no encuentra la action |
| `uses: ./` | "usa la action que está en la raíz de este repo" |
| `with:` | le pasas los inputs |
| `id: hola` | nombre del step, para leer su output después |
| `steps.hola.outputs.time` | el output `time` del step `hola` |

### 6 Subir y comprobar

```
git add . && git commit -m
"Añade action Docker y workflow"
git remote add origin https://github.com/
fabiangorbaranaquino/hola-action.git
git push -u origin main
```

✅ Actions → "Usar mi action": un step que construye la imagen Docker, otro que dice **Hola Fabián** y el último **Hora del saludo …** con la fecha.

❌ "permission denied" → olvidaste `chmod +x entrypoint.sh`: hazlo, `git commit -am "Hace ejecutable el script"`, push.

### 🟡 H1 · Ejercicio

1. ☐ Llama a tu action dos veces en el mismo job, con nombres distintos.
2. ☐ Llámala sin `with`: ¿qué nombre saluda? ¿Por qué?

### 🔴 H2 · Ejercicio

1. ☐ Añade un input `idioma` (no obligatorio, por defecto `es`) y que diga "Hola" o "Hello" según el valor.

Solución → SH

👹 EXAMEN FINAL SIMULACRO

# Proyecto cafetería

⏱ 60 min | 📄 Con la chuleta, como el real | ⏱ Cronométrate

Hazlo de principio a fin sin mirar este PDF. Solo la chuleta. Al acabar compara con la solución.

### 0 · Repositorio

Crea `~/examen2/cafeteria` como repo con rama `master`. Crea `Readme.md` con "# Cafetería" y haz el primer commit.

### 1 · Ignorar ficheros

Haz que Git ignore todos los `.log` y el fichero `secretos.env`. Crea `error.log` y `secretos.env` y demuestra con `git status` que no aparecen. Commit.

### 2 · Rama carta

Crea y muévete a la rama `carta`. Crea `index.html` con un HTML básico (commit) y añade una sección `<h2>Carta</h2>` en **otro** commit.

### 3 · master avanza

Vuelve a `master` sin fusionar. Crea `horario.html` con una línea de contenido. Commit. Enseña el gráfico donde se vea que divergen.

### 4 · Fusión

Fusiona `carta` en `master`. ¿Fast-forward o no? ¿Por qué?

### 5 · Conflicto

Crea la rama `titulo`, cambia el `<h1>` de `index.html` a "Cafetería Fabián" y commit. En `master`, cambia la misma línea a "Bienvenidos" y commit. Fusiona `titulo` en `master` y resuelve dejando "Bienvenidos a Cafetería Fabián".

### 6 · Versión

Crea en master el tag **anotado** `v1.0` con mensaje.

### 7 · Gitflow

Crea `develop` desde master. Haz la feature `feature/reservas` (añade `reservas.html`) y ciérrala en develop con `--no-ff`, borrando la rama.

### 8 · Web

Desde `master`, crea la rama `gh-pages` con un `style.css` que ponga el `h1` en marrón y enlázalo en el `index.html`. Commit.

### 9 · Workflow

En `master`, crea `.github/workflows/ci.yml` que se ejecute en push a `master` y `develop` y que haga checkout, muestre "Cafetería OK" y liste los ficheros. Commit.

### 10 · Limpieza

Borra las ramas `carta` y `titulo`.

### 11 · Entrega

Crea en GitHub el repo `cafeteria` vacío. Sube `master`, `develop`, `gh-pages` y el tag. Activa Pages desde `gh-pages`. Comprueba: Actions con el workflow ✅ en master y la web con el título marrón.

**🎯 Comprobación final**

`git log --oneline --graph --all --decorate` → ramas master, develop y gh-pages · tag v1.0 · sin carta ni titulo · todo subido.

## 🏆 SI TE SALE ESTO, TE SALE EL EXAMEN

Repasa solo en lo que hayas fallado. Y al examen, tranquilo: lleva la chuleta y antes de cada paso → git status + git branch.

SOLUCIONES

# Ejercicios y examen

## SA · Workflow desde cero

```
cd ~/git
mkdir repaso-actions && cd repaso-actions
git init && git branch -m main
mkdir -p .github/workflows
cat > .github/workflows/hola.yml << 'EOF'
name: Primer workflow
on: [push]
jobs:
  saludar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hola"
EOF
cat .github/workflows/hola.yml
git add .github && git commit -m "Añade workflow"
git remote add origin https://github.com/
fabiangorbaranaquino/repaso-actions.git
git push -u origin main
# A2: nano → añadir al final:
      - run: |
          date
          ls -la
git commit -am "Añade fecha y listado" && git push
```

## SB · Filtros

1. `git switch main && git switch -c develop && git push -u origin develop` → **no** se ejecuta: el .yml de develop (copiado de main) solo tiene `[ main ]` (o `[ main, prueba ]`).
2. En develop: `nano` → `branches: [ main, develop ]` → `git commit -am "Añade develop al filtro" && git push` → se ejecuta.
3. `git switch main && cat .github/workflows/hola.yml` → no tiene develop: el cambio solo se hizo en la rama develop.

## SC · euro.py extra

```
git switch main && git switch -c gbp
nano .github/workflows/actions.yml
#   branches: [ main, gbp ]
#   python-version: '3.11'
#   run: python ./euro.py GBP
git commit -am "Rama gbp con GBP y Python 3.11"
git push -u origin gbp
git switch development
nano .github/workflows/actions.yml     # run: python ./
euro.py USD JPY
git commit -am "Development muestra USD y JPY" && git
push
```

## SD · Gitflow D1

```
git switch -c feature/carrito develop
echo "<h1>Carrito</h1>" > carrito.html
git add carrito.html && git commit -m "Añade carrito"
git switch develop
git merge --no-ff feature/carrito -m "Merge feature/
carrito"
git branch -d feature/carrito

git switch -c release/1.1.0 develop
echo "1.1.0" > VERSION && git commit -am "Prepara
versión 1.1.0"
git switch master
git merge --no-ff release/1.1.0 -m "Merge release/
1.1.0"
git tag -a 1.1.0 -m "Versión 1.1.0"
git switch develop
git merge --no-ff release/1.1.0 -m "Merge release/1.1.0
en develop"
git branch -d release/1.1.0

git switch -c hotfix/1.1.1 master
echo "fix" >> carrito.html && git commit -am "Arregla
carrito"
git switch master
git merge --no-ff hotfix/1.1.1 -m "Merge hotfix/1.1.1"
git tag -a 1.1.1 -m "Versión 1.1.1"
git switch develop
git merge --no-ff hotfix/1.1.1 -m "Merge hotfix/1.1.1
en develop"
git branch -d hotfix/1.1.1
```

## SG · matrix y secrets

```
# matrix.yml
    strategy:
      matrix:
        python: ['3.11', '3.12', '3.13']
    runs-on: ubuntu-latest
# → 3 jobs (1 sistema × 3 versiones)

# secreto.yml (crea antes USUARIO_BD en Settings →
Secrets)
      - env:
          CLAVE: ${{ secrets.MI_CLAVE }}
          USUARIO: ${{ secrets.USUARIO_BD }}
        run: echo "Usuario $USUARIO, clave $CLAVE"
# en el log: Usuario ***, clave ***
```

## SH · Custom action

```
# H1 (en usar.yml)
      - uses: ./
        with:
          who-to-greet: 'Fabián'
      - uses: ./
        with:
          who-to-greet: 'Mona'
      # sin with → "Hola World" (usa el default)
      - uses: ./

# H2 · action.yml: nuevo input y nuevo arg
  idioma:
    description: 'es o en'
    required: false
    default: 'es'
  args:
    - ${{ inputs.who-to-greet }}
    - ${{ inputs.idioma }}

# entrypoint.sh
#!/bin/sh -l
if [ "$2" = "en" ]; then echo "Hello $1"; else echo
"Hola $1"; fi
echo "time=$(date)" >> $GITHUB_OUTPUT

# usar.yml
        with:
          who-to-greet: 'Mona'
          idioma: 'en'
```

# 👹 Examen final

```
# 0
mkdir -p ~/examen2/cafeteria && cd ~/examen2/cafeteria
git init && git branch -m master
echo "# Cafetería" > Readme.md
git add Readme.md && git commit -m "Primer commit:
añade Readme"
# 1
printf "*.log\nsecretos.env\n" > .gitignore
touch error.log secretos.env
git status                      # solo sale .gitignore
git add .gitignore && git commit -m "Ignora logs y
secretos"
# 2
git switch -c carta
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>Cafetería</title></
head>
<body>
<h1>Cafetería</h1>
</body>
</html>
EOF
git add index.html && git commit -m "Añade index base"
nano index.html                 # debajo del h1:
<h2>Carta</h2>
git commit -am "Añade sección carta"
# 3
git switch master
echo "<p>Abierto de 8 a 20</p>" > horario.html
git add horario.html && git commit -m "Añade horario"
git log --oneline --graph --all
# 4 (NO fast-forward: las dos ramas avanzaron)
git merge carta -m "Merge branch 'carta'"
git log --oneline --graph --all
# 5
git switch -c titulo
nano index.html                 # <h1>Cafetería
Fabián</h1>
git commit -am "Título Cafetería Fabián"
git switch master
nano index.html                 # <h1>Bienvenidos</h1>
git commit -am "Título Bienvenidos"
git merge titulo                # CONFLICT
```

```
nano index.html                 # <h1>Bienvenidos a
Cafetería Fabián</h1>, sin marcas
git add index.html && git commit -m "Resuelve conflicto
del título"
# 6
git tag -a v1.0 -m "Versión 1.0"
# 7
git switch -c develop master
git switch -c feature/reservas develop
echo "<h1>Reservas</h1>" > reservas.html
git add reservas.html && git commit -m "Añade reservas"
git switch develop
git merge --no-ff feature/reservas -m "Merge feature/
reservas"
git branch -d feature/reservas
# 8
git switch master
git switch -c gh-pages
echo "h1 { color: brown; }" > style.css
nano index.html                 # en <head>: <link
rel="stylesheet" href="style.css">
git add style.css index.html && git commit -m "Añade
estilos para Pages"
# 9
git switch master
mkdir -p .github/workflows
cat > .github/workflows/ci.yml << 'EOF'
name: CI cafetería
on:
  push:
    branches: [ master, develop ]
jobs:
  comprobar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Cafetería OK"
      - run: ls -la
EOF
git add .github && git commit -m "Añade workflow CI"
# 10
git branch -d carta titulo
# 11
git remote add origin https://github.com/
fabiangorbaranaquino/cafeteria.git
git push -u origin master
```

```
git push -u origin develop
git push -u origin gh-pages
git push origin --tags
# Settings → Pages → gh-pages / root → Save
git log --oneline --graph --all --decorate
```

Nota: el workflow está en master, así que se ejecuta en el push de master. En develop no se ejecuta porque develop salió de master **antes** de crear el ci.yml (no lo tiene). Si quieres que también corra en develop: `git switch develop && git merge master -m "Trae workflow" && git push`.